HWRS640 — Assignment 4

Streamflow Prediction with an LSTM

Hossein Yousefi Sohi — `hyousefi@arizona.edu` Spring 2026

Repository: https://github.com/hyousefi98/hwrs640_HW4_HY

---

1. What I built

I trained a single LSTM to predict daily streamflow for the 50 basins in the `minicamels` dataset. The input is 365 days of past weather plus 14 static basin attributes. The output is tomorrow's streamflow.

The whole project is a small Python package (`camelflow`) with a CLI:

```
camelflow summarize-data   # exploratory plots
camelflow train            # train the model
camelflow evaluate         # score a checkpoint on the test
set
camelflow plot             # make every report figure
```

Setup, hyperparameters, and reproduction steps are in the `README.md`. The rest of this document answers the four assignment problems.

2. Problem 1 — Looking at the data

What's in the dataset

basins	50
time span	1980-10-01 to 2010-09-30 (30 water years, 10,957 days)
daily forcings	`prcp` , `tmax` , `tmin` , `srad` , `vp`
target	`qobs` (observed streamflow, mm/day)
static attributes used	14

The 14 static attributes cover elevation, slope, area, long-term climate indices, baseflow index, snow fraction, soil depth, forest cover, and LAI.

How I split the data

Every basin is in every split. I split only by time. Training covers water years 1981 through 2000, validation 2001 through 2005, and test 2006 through 2010.

This is the standard CAMELS benchmark split. The model sees all basins during training, but the years used for validation and testing are unseen. Splitting strictly by time means the model can't look at future labels. The 365-day window for the first test sample is allowed to reach back into the validation period, but only for forcings. Labels are never shared across splits.

What a training sample is

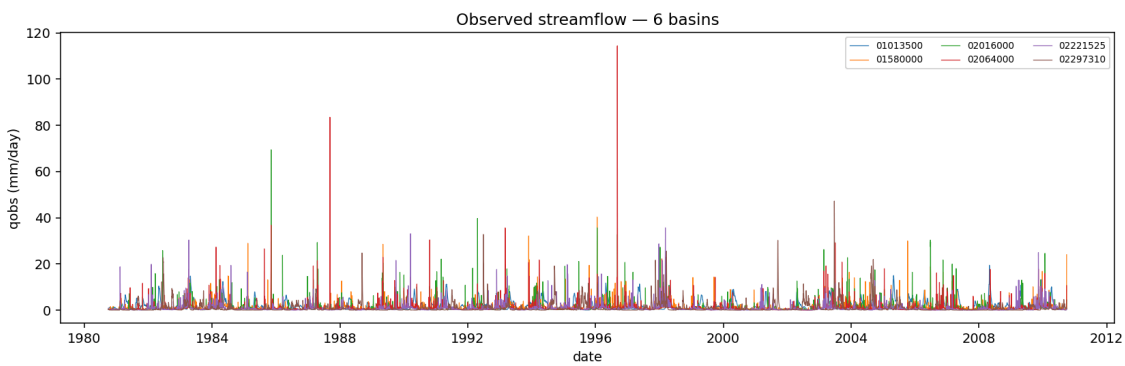
One sample is 365 days of the five forcings, plus the 14 static attributes for that basin. The target is observed streamflow on the day after the window. So the model forecasts one day ahead from a rolling year of history.

Preprocessing

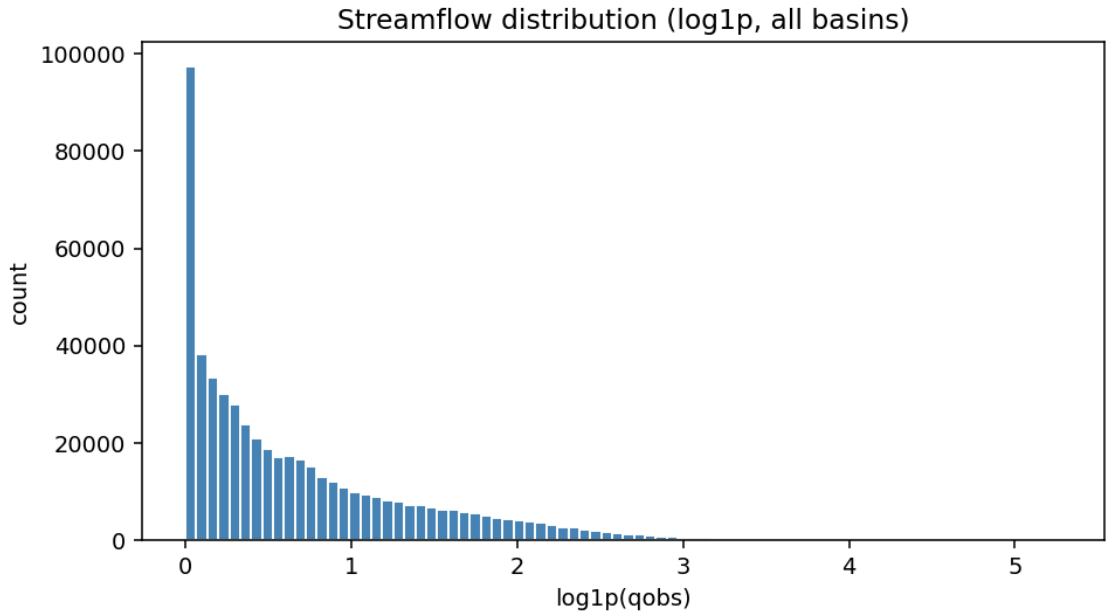
I normalize with training-period statistics only. Forcings and static attributes get a z-score. A few static attributes have gaps, so I fill them with the median before z-scoring. Streamflow spans orders of magnitude, so I apply `log1p` to the target and then z-score it. Metrics are computed after inverting both transforms. Samples with NaN in the window or the label are dropped during indexing.

Exploratory plots

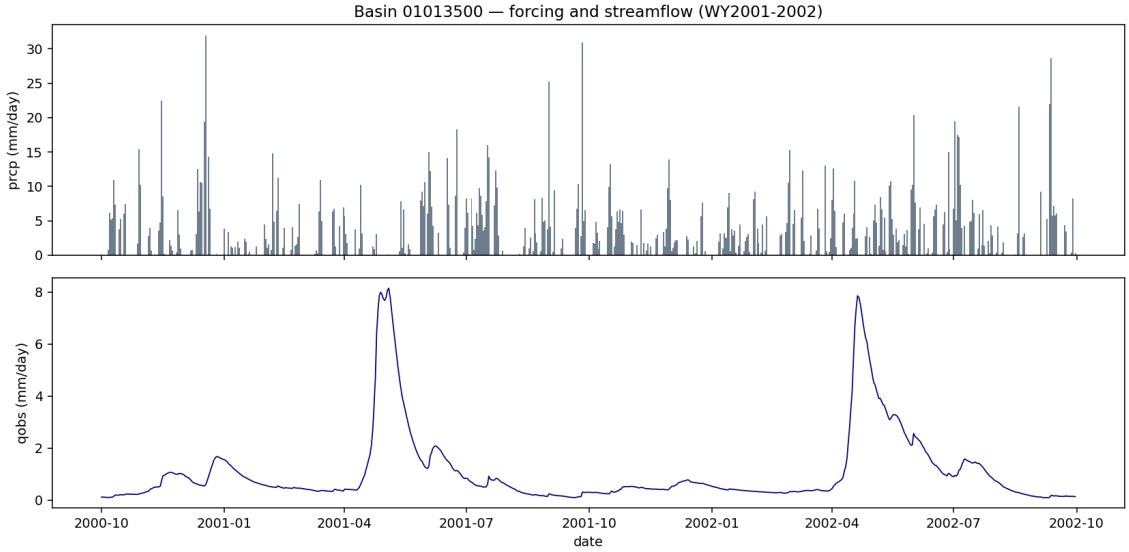
Streamflow for six basins over the full 30 years:



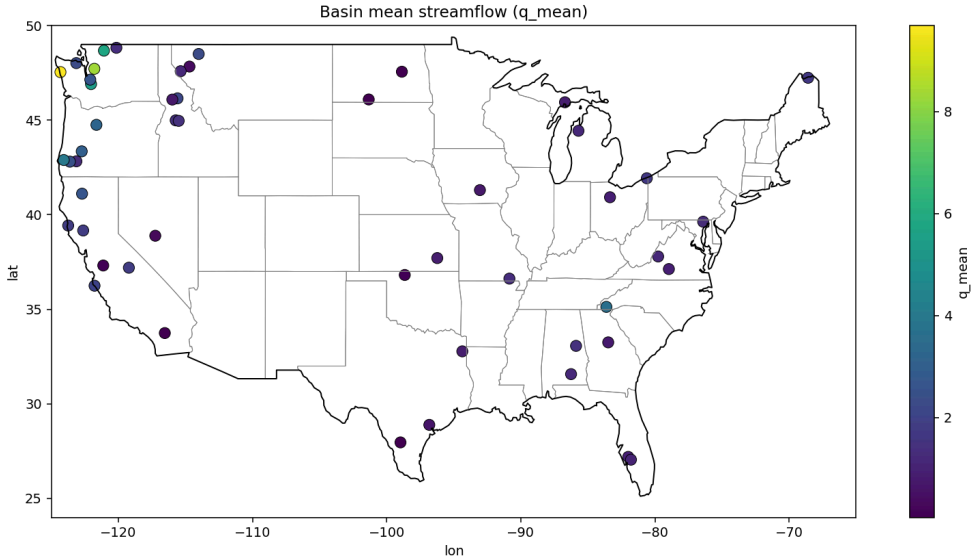
Streamflow is very heavy-tailed, which is why the target is log-transformed during training:



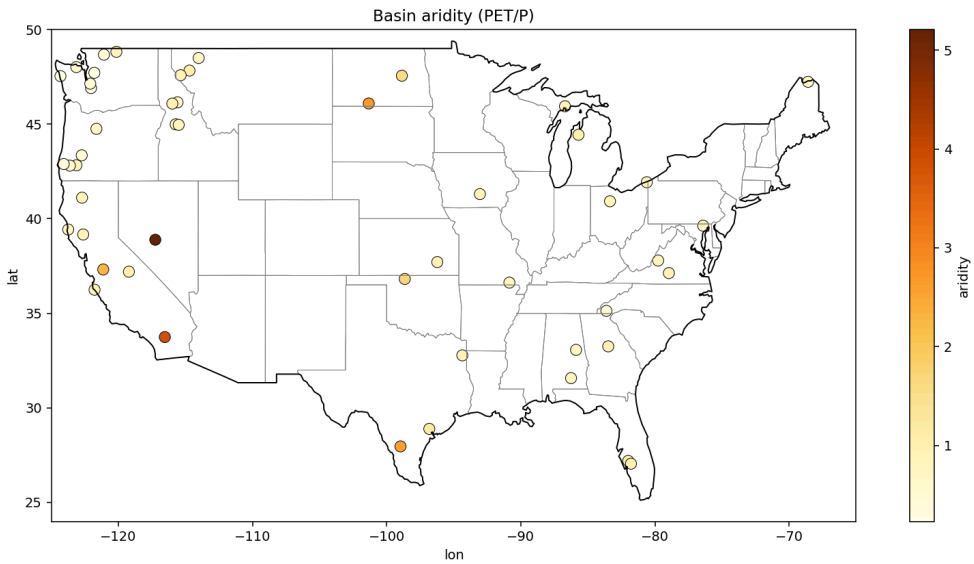
Precipitation and streamflow for one basin over two water years:



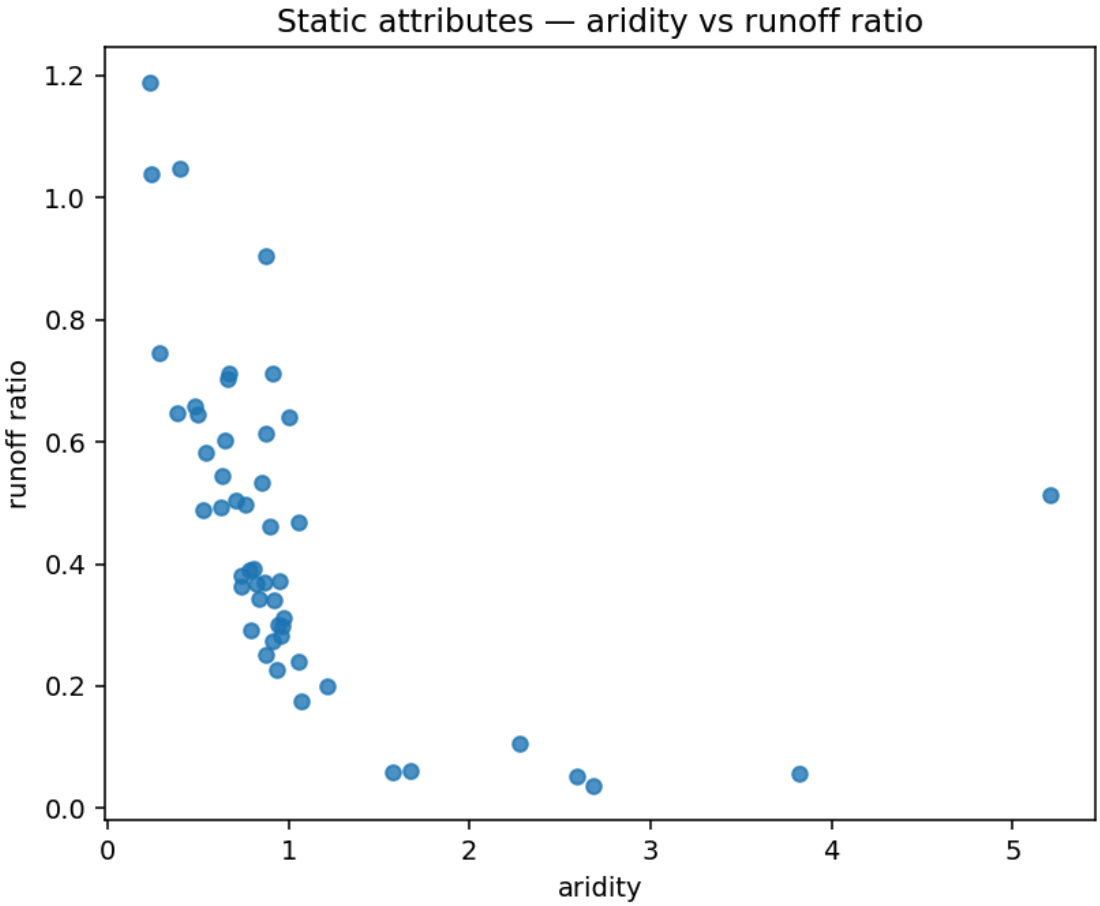
Basin locations and mean flow across CONUS:



The same map colored by aridity:



Aridity against runoff ratio across the 50 basins:



3. Problem 2 — Model

Architecture

```
input (batch, 365, 5 forcings + 14 static attributes)
      │
      ▼
LSTM  (hidden = 128, 1 layer, dropout = 0.5)
      │     take the last hidden state
      ▼
Dropout → Linear(128 → 1)
      │
      ▼
standardized log-flow  →  inverse transform  →  predicted
qobs
```

The 14 static attributes are broadcast across all 365 timesteps and concatenated with the five forcings, so the LSTM input is 19-dimensional. Only the final hidden state feeds the linear head. The model has about 76,000 parameters.

Why this architecture

Streamflow on a given day depends on weeks or months of previous weather. An LSTM handles that kind of long memory well, and it has been shown to work on CAMELS in the Kratzert papers. Training one model on all 50 basins is more data-efficient than fitting one model per basin; the static attributes tell the network which basin it is looking at.

One strength and one weakness

Strength. Skill transfers between basins with similar attributes. Improvements on one snow-dominated basin help the other snow-dominated basins in the dataset.

Weakness. The head predicts one day ahead and only gives a point estimate. There is no uncertainty, and a longer forecast horizon would need an autoregressive or sequence-to-sequence extension.

4. Problem 3 — Training pipeline

Training logic is in `src/camelflow/train.py`, wired up to the CLI in `src/camelflow/cli.py`. Command used for the reported run:

```
camelflow train --seq-len 365 --epochs 50 --batch-size 256 \
                --hidden 128 --dropout 0.5 \
                --lr 1e-3 --weight-decay 1e-5 \
                --loss nse --patience 8 --seed 42
```

Optimizer is Adam with weight decay. Learning rate is halved by `ReduceLROnPlateau` after three epochs without improvement in validation loss. Gradient norm is clipped at 1.0. Training early-stops after eight epochs without improvement in validation NSE. The best checkpoint (on val NSE) and the last checkpoint are saved separately.

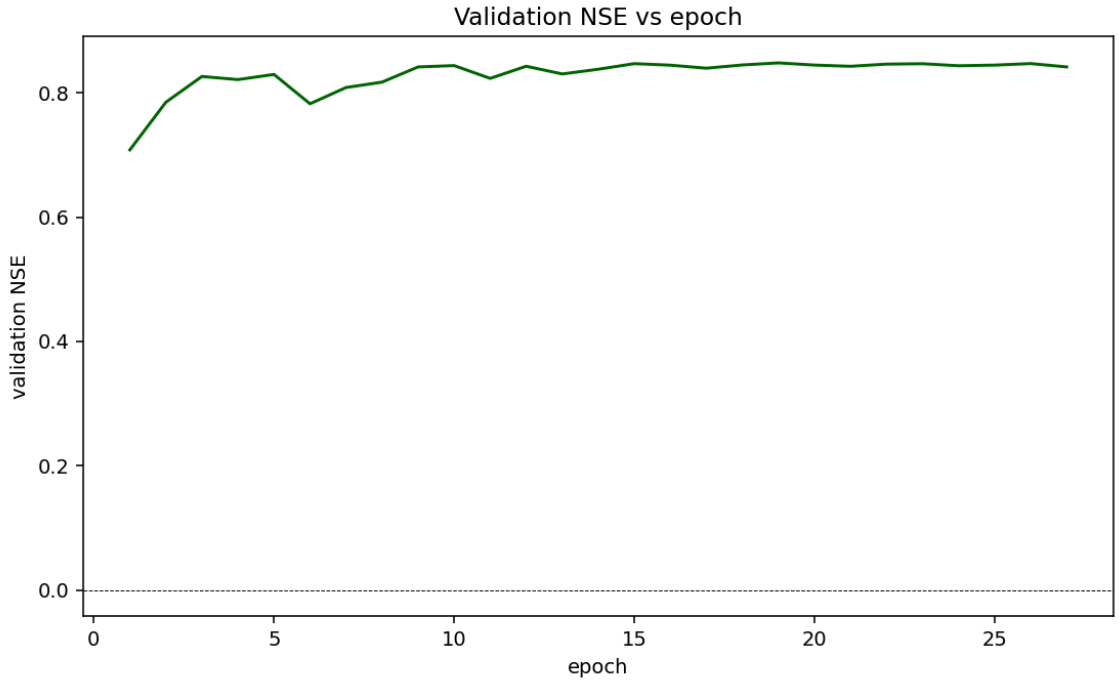
Loss is the basin-averaged NSE* loss from Kratzert et al. 2019:

$$\mathcal{L} = \frac{1}{N} \sum_{n} \frac{(\hat{y}_n - y_n)^2}{(\sigma_{b(n)} + \epsilon)^2}$$

σ_b is the training-period standard deviation of the target for basin b. Basins with tight flow distributions are no longer drowned out by large errors on wet, high-flow basins. An earlier run with plain MSE gave median test NSE 0.77; the same config with NSE* loss plus weight decay gave 0.79, and lifted the worst basin from NSE = −0.68 up to +0.30.

Training and validation curves

Training early-stopped at epoch 27. Best validation NSE was 0.848 at epoch 19. Total wall time was about 25 minutes on an RTX 6000 Ada.



Training loss keeps dropping after validation plateaus. Dropout at 0.5 and weight decay of 1e-5 slow this down but don't eliminate it, so early stopping is what actually sets the final checkpoint.

5. Problem 4 — Evaluation and interpretation

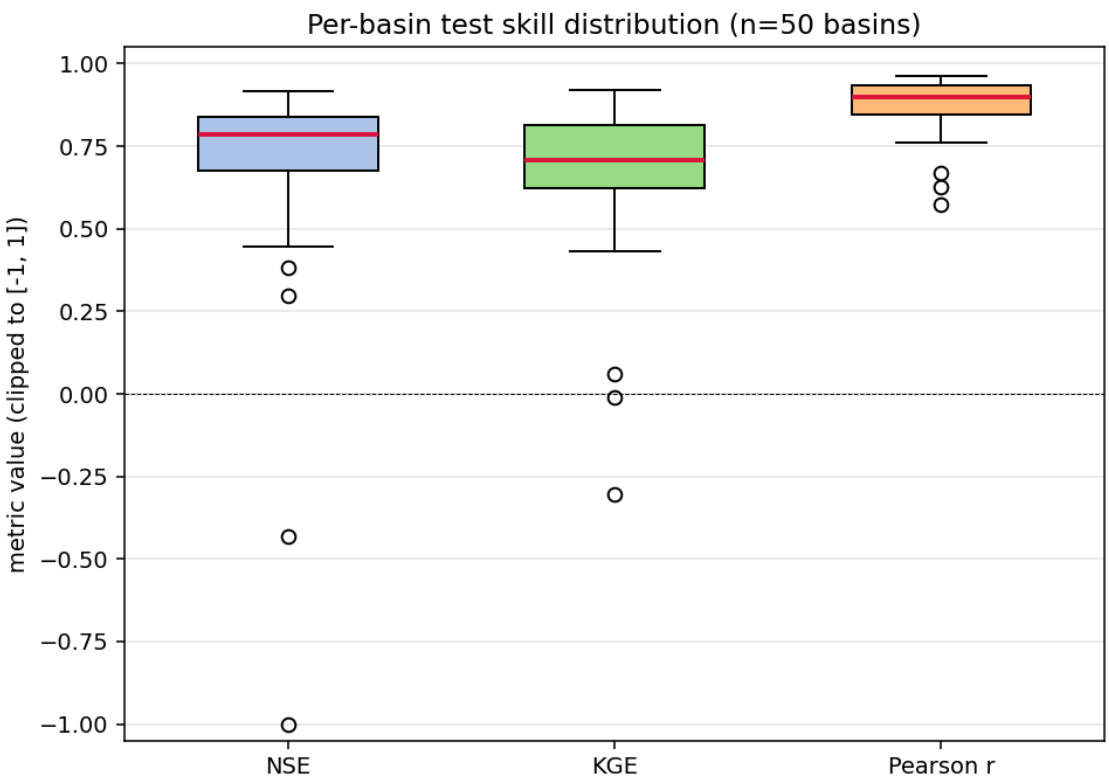
Test metrics on WY2006–2010, 50 basins

metric	median across basins
NSE	0.786
KGE	0.709
RMSE	1.07 mm/day
Pearson r	0.88
peak-flow bias (q98)	slight under-prediction

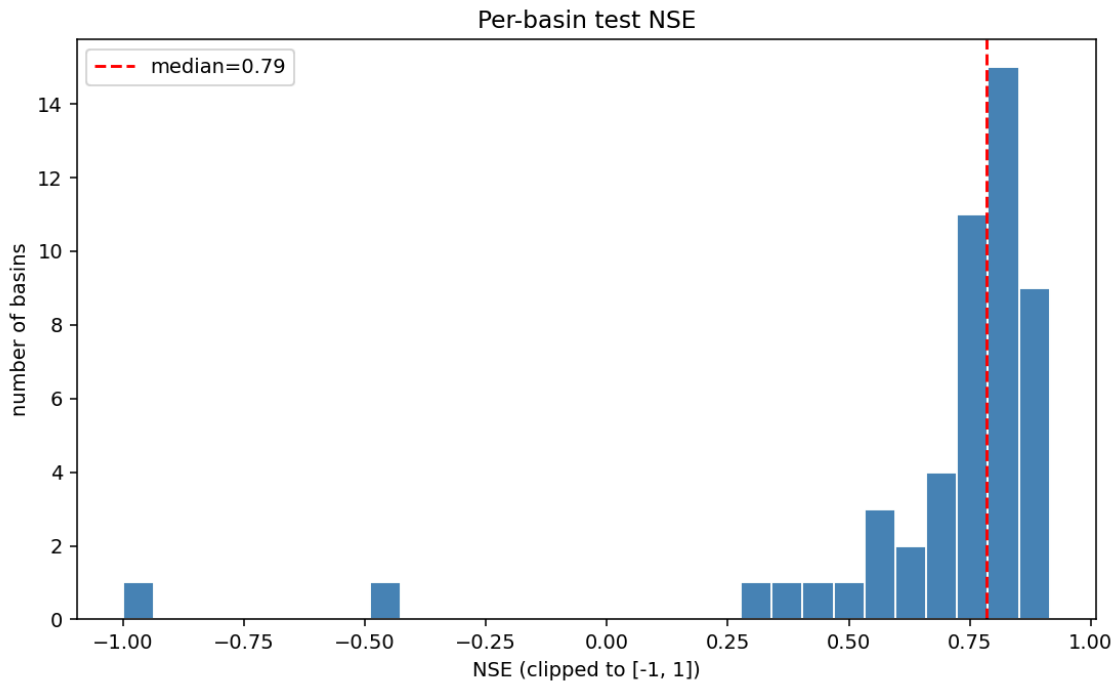
48 of 50 basins have a positive NSE. Per-basin values are in `outputs/test_metrics.csv`.

Distribution of skill across basins

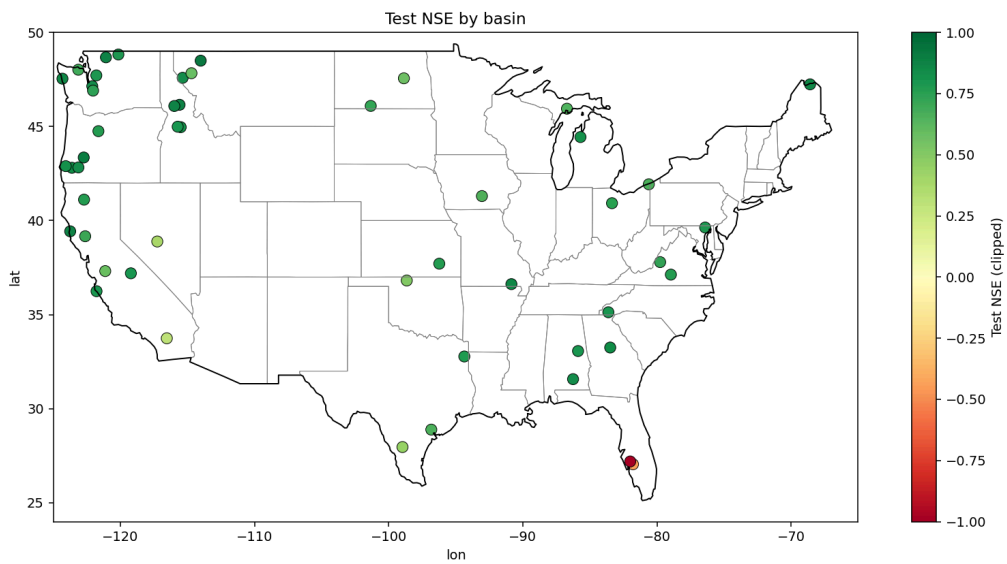
Boxplot of NSE, KGE, and Pearson r:



Histogram of per-basin test NSE:



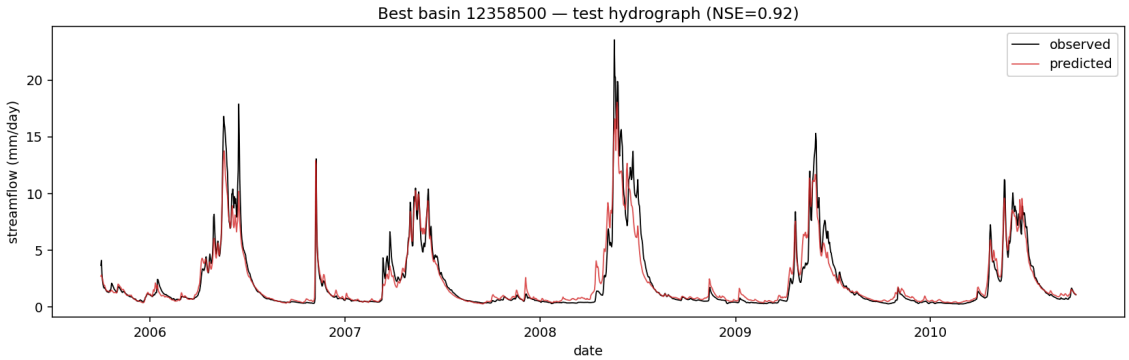
Basin map colored by test NSE:

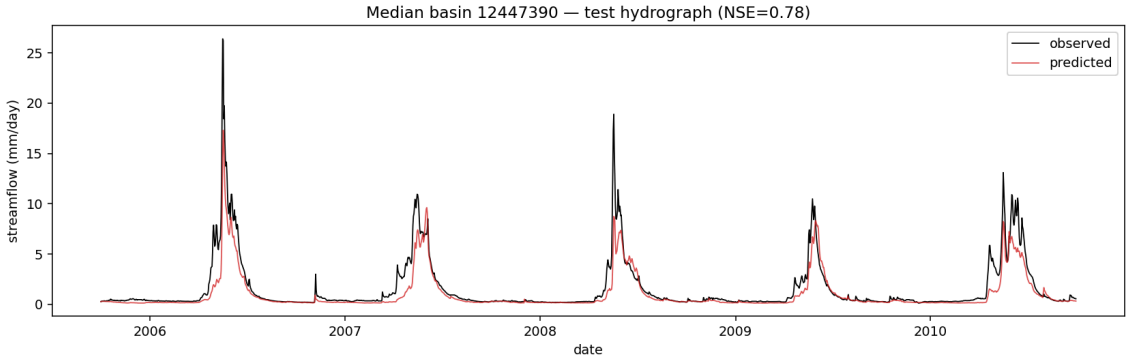


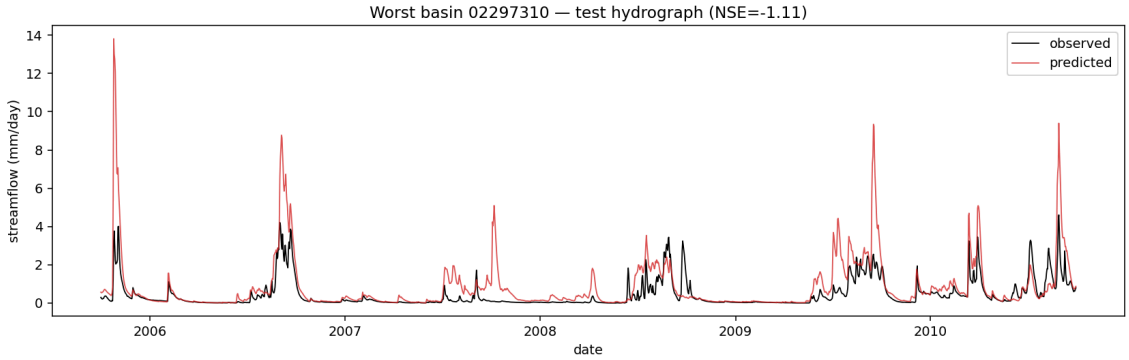
Skill is high in the Pacific Northwest, the Northern Rockies, and the Northeast. It drops in the arid Southwest and in the Gulf coast and Florida basins.

Observed vs predicted streamflow

Three basins are shown: the best NSE, the basin closest to the median NSE, and the worst.

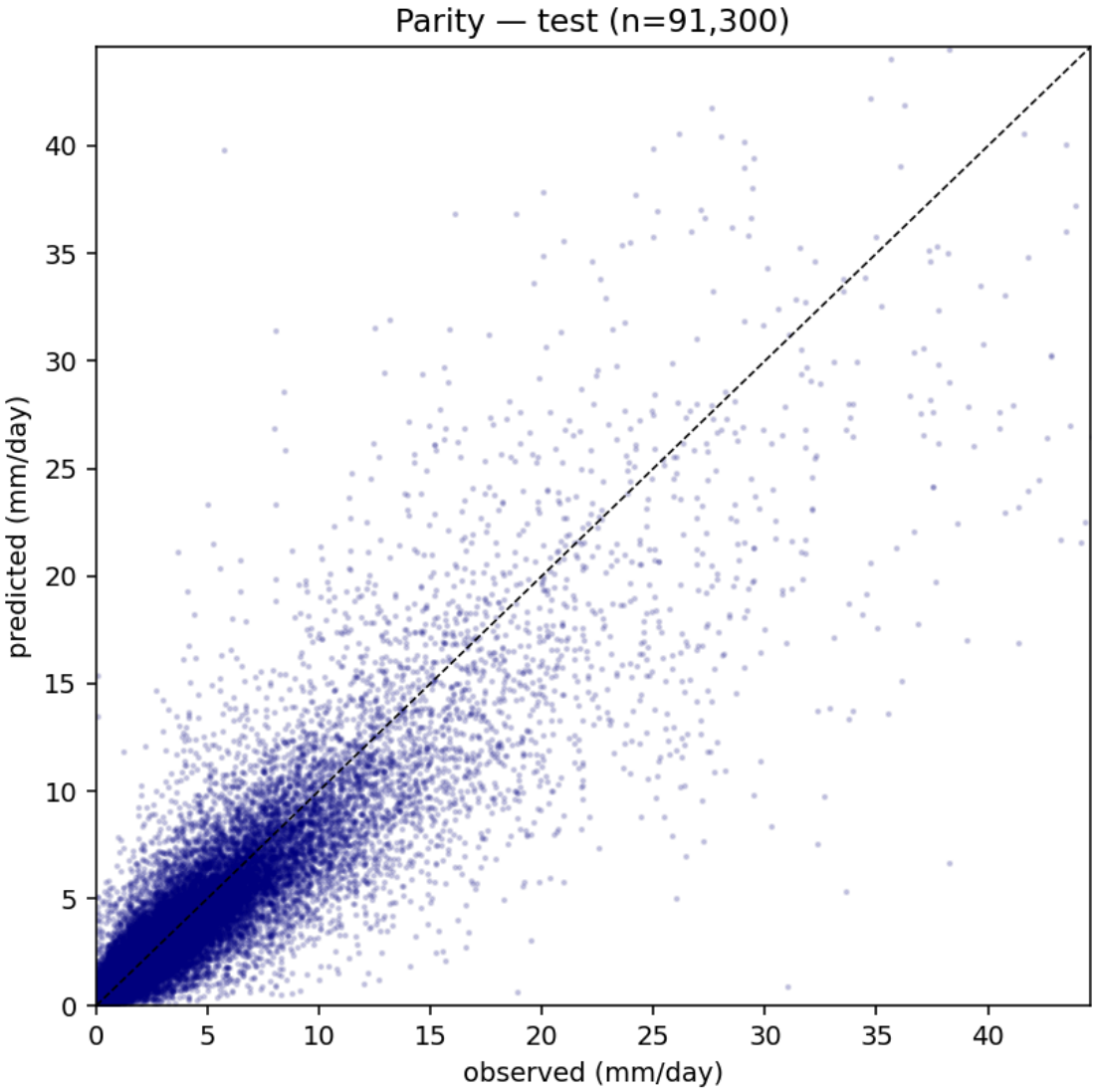






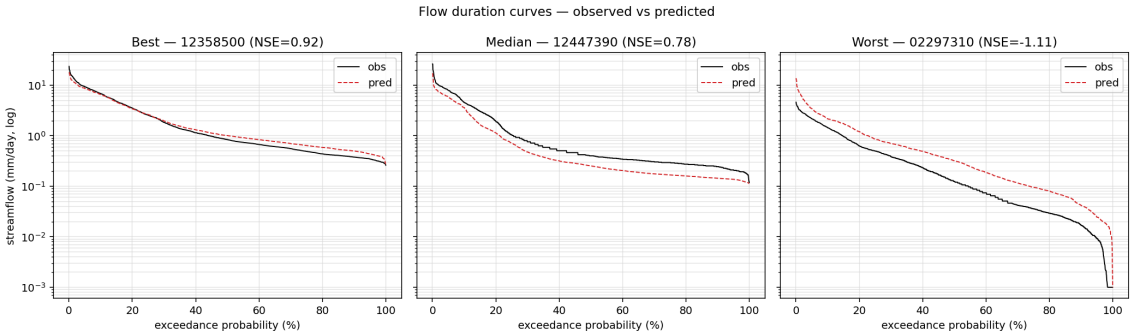
Parity plot

All test samples, predicted against observed, with the 1:1 line:



Flow-duration curves

FDCs show whether the model reproduces the distribution of flows, not just their timing:



For the best and median basins the predicted and observed FDCs track each other across the whole range. For the arid basin the predicted FDC is flatter than

the observed one; the model under-predicts the high flows and over-predicts the low flows.

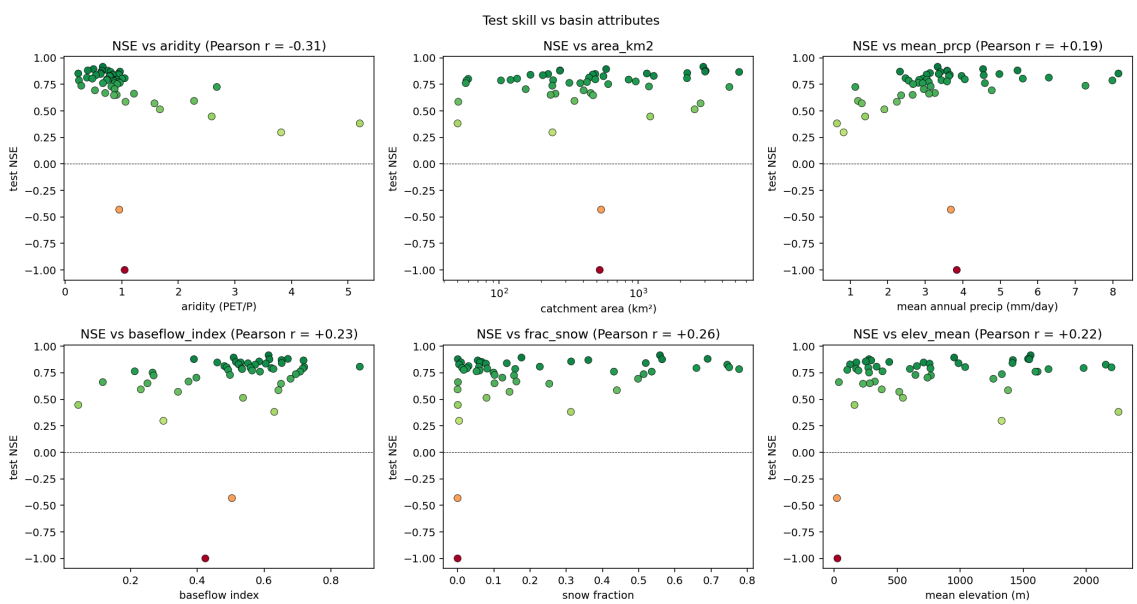
Best and worst basin

Best: `12358500`, Middle Fork Flathead River near West Glacier, MT. NSE = 0.92, KGE = 0.86, r = 0.96. A large, snow-dominated mountain basin with a strong seasonal cycle. The LSTM captures both the spring melt and the summer recession.

Worst: `02297310`, Horse Creek near Arcadia, FL. NSE = −1.11. A low-relief subtropical basin where most streamflow comes from individual convective storms. The hydrograph responds in hours, but the forcings only resolve daily totals. The model's prediction is a smoothed and delayed version of the observed hydrograph.

Why skill varies across basins

Test NSE plotted against each static attribute:



Two patterns show up in this panel:

1. Skill drops as aridity increases. Dry basins have small runoff coefficients and their responses are dominated by sub-daily events that the forcings do not resolve.
2. Skill rises with baseflow index and with snow fraction. Slow groundwater-fed and snowmelt-driven basins have a seasonal structure that a 365-day LSTM picks up easily.

There is also a mild positive trend with elevation and a weak trend with basin area. These trends line up with the geographic pattern in the NSE map: good performance in the snow-dominated and humid basins of the Pacific Northwest and Rockies, weaker performance in the arid Southwest and the flashy coastal plains.

A closer look at the two worst basins

Pulling the attributes of the two basins at the bottom of the test-NSE ranking makes the pattern even clearer:

basin_id	test NSE	elev_mean (m)	frac_snow	aridity	mean_prcp (mm/day)
02298123 — Prairie Creek, FL	−0.43	22	0.00	0.96	3.68
02297310 — Horse Creek, FL	−1.11	26	0.00	1.05	3.84

For comparison, the three best basins all sit above 950 m with snow fractions between 0.18 and 0.69. The two worst basins are at roughly sea level and receive essentially no snow.

Two things follow from these numbers. Without snowpack the basins lose the biggest source of temporal smoothing in a hydrologic system. Snow accumulates precipitation over months and releases it as a slow seasonal signal that an LSTM with a 365-day window can pick up. A basin with `frac_snow = 0` has no such signal. And because the elevation is near sea level the terrain is flat, flow paths are short, and storage delays are small, so the hydrograph responds to individual storms on timescales of hours. Daily forcings cannot resolve that behavior, so there is a floor on how accurate the model can be on these basins regardless of architecture.

These basins fail for a data reason more than a model reason. They are the wrong fit for daily forcings and a one-day-ahead point-estimate model. Improving them would take sub-daily forcings or a different class of model that can represent event-driven runoff explicitly.

6. What worked, what didn't, what I'd try next

What worked. The standard CAMELS-LSTM setup — one layer, 365-day window, dynamic and static inputs combined at the input — gave a reasonable baseline right away. Switching the loss from MSE to the basin-NSE* loss, together with weight decay and dropout 0.5, was the biggest improvement: median test NSE went from 0.77 to 0.79, and the worst basin moved from NSE = −0.68 to +0.30.

What didn't. Arid and flashy basins remain the weakness. Horse Creek FL, Prairie Creek FL, and Murder Creek GA are places where most streamflow comes from individual storm events. Daily forcings cannot resolve those events. A visible train vs val gap also persists after the LSTM has converged, which suggests either the model is slightly too large for 20 years of data per basin, or there is more regularization to be found.

What I would try next with more time.

- Sub-daily forcings for the flashy basins, or a separate event-driven model on top of the LSTM.
- The EA-LSTM from Kratzert et al. 2019. Static attributes gate the input rather than being concatenated.
- Ensembling over a few seeds to produce uncertainty bounds on the predictions.

How to reproduce

```
git clone https://github.com/hyousefi98/hwrs640_HW4_HY
cd hwrs640_HW4_HY
uv sync

uv run camelflow train --epochs 50 --seq-len 365 --batch-size 256 \
                       --hidden 128 --dropout 0.5 --weight-decay
1e-5 \
                       --loss nse --lr 1e-3 --seed 42
uv run camelflow plot --checkpoint outputs/best.pt
```

References

- Kratzert, F. et al. (2018). Rainfall–runoff modelling using LSTM networks. *HESS* 22, 6005–6022.
- Kratzert, F. et al. (2019). Towards learning universal, regional, and local hydrological behaviors via ML applied to large-sample datasets. *HESS* 23, 5089–5110.
- Newman, A. J. et al. (2015). Development of a large-sample watershed-scale hydrometeorological data set for the contiguous USA. *HESS* 19, 209–223.
- Addor, N. et al. (2017). The CAMELS data set. *HESS* 21, 5293–5313.
- `minicamels` — https://github.com/BennettHydroLab/minicamels